ENTERPRISE-GRADE THREAT DETECTION & AUTOMATED RESPONSE PLATFORM

Implementation Report — Lab Deployment, Simulated Attack & Incident Response

Using Wazuh, MITRE ATT&CK and Manual Active Response

Younes Belayouni

July 27, 2026

Cybersecurity Engineering Report

1. Introduction

Modern organizations face a growing volume and sophistication of cyberattacks. A Security Operations Center (SOC) is responsible for continuously monitoring, detecting, and responding to security incidents. This report documents the practical build-out of a scaled-down, self-hosted SOC lab that mirrors the architecture used by Managed Security Service Providers (MSSPs), using the open-source Wazuh platform.

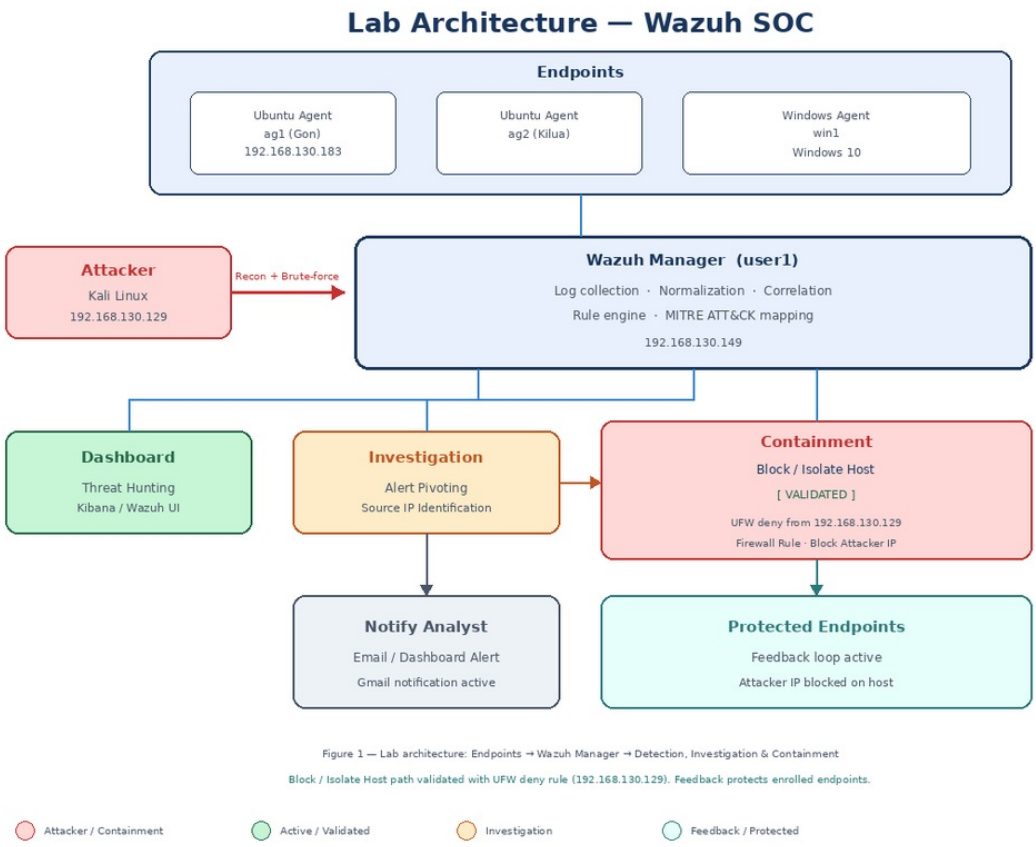
Rather than presenting only a theoretical design, this document walks through the actual steps taken: deploying and enrolling endpoints, provisioning lab user accounts, launching a real reconnaissance and brute-force attack from a Kali Linux machine, detecting that attack through Wazuh's correlation engine, investigating it in the Threat Hunting module, and finally containing it with a firewall rule — evidenced with dashboard and terminal captures collected throughout the exercise.

2. Project Objectives

- Deploy a centralized log collection and analysis platform using Wazuh.
- Enroll heterogeneous endpoints (Linux and Windows) as monitored agents.
- Simulate a real-world brute-force attack from an external attacker machine (Kali Linux).
- Detect the attack in real time and map it to the MITRE ATT&CK framework.
- Investigate the alert chain in the Threat Hunting module to identify the attacker's source IP.
- Contain the threat using an Active Response firewall rule.
- Document the complete attack-to-response chain with evidence.

3. System Architecture

The platform follows a layered SOC design: monitored endpoints feed the Wazuh manager, which normalizes and correlates events, escalates high-severity alerts, and exposes them through a Kibana-based dashboard for investigation. Figure 1 below reflects the architecture as it was actually deployed and exercised in this lab, including the naming conventions used for the manager and the two lab user accounts, the attacker path, and the validated containment step.



Compared to the original conceptual design, the diagram highlights the concrete components validated in this exercise: the Wazuh Manager is identified as “user1”; the Endpoints layer explicitly lists the Ubuntu agents ag1 (“Gon”) and ag2 (“Kilua”) together with the Windows agent “win1”; and the attacker (Kali Linux, 192.168.130.129) plus the full detection → investigation → containment path are drawn explicitly. Network devices are shown as supporting infrastructure that can feed the manager. Threat-intelligence enrichment and SOAR layers remain outside the current validated scope of this lab.

4. Deployment and Agent Enrollment

The Wazuh manager, indexer and dashboard were deployed first, after which endpoints were enrolled as agents. A Windows 10 host was enrolled by downloading the MSI agent package and installing it silently with the manager address and agent name specified, then starting the Wazuh service.

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi
-OutFile $env:tmp\wazuh-agent;
msiexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.130.149'
WAZUH_AGENT_NAME='win1'
NET START Wazuh
```

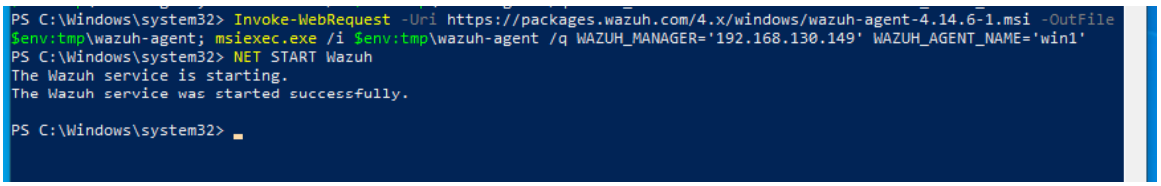


Figure 2 — Silent installation and startup of the Wazuh agent on the Windows endpoint (win1).

Once both endpoints were registered, the Endpoints view confirmed two enrolled agents: the Ubuntu host (“user1”) and the Windows host (“win1”), the latter reporting as active.

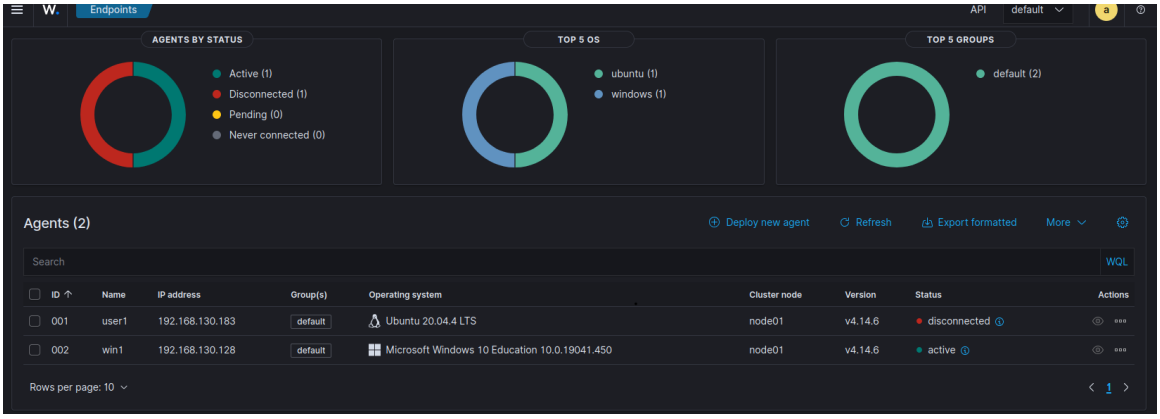


Figure 3 — Endpoints overview showing both enrolled agents, operating systems, and connection status.

Drilling into the Windows agent's detail view confirmed system inventory (CPU, memory, cores), MITRE ATT&CK tactic tagging, and PCI DSS compliance scoring were all being collected correctly for that host.

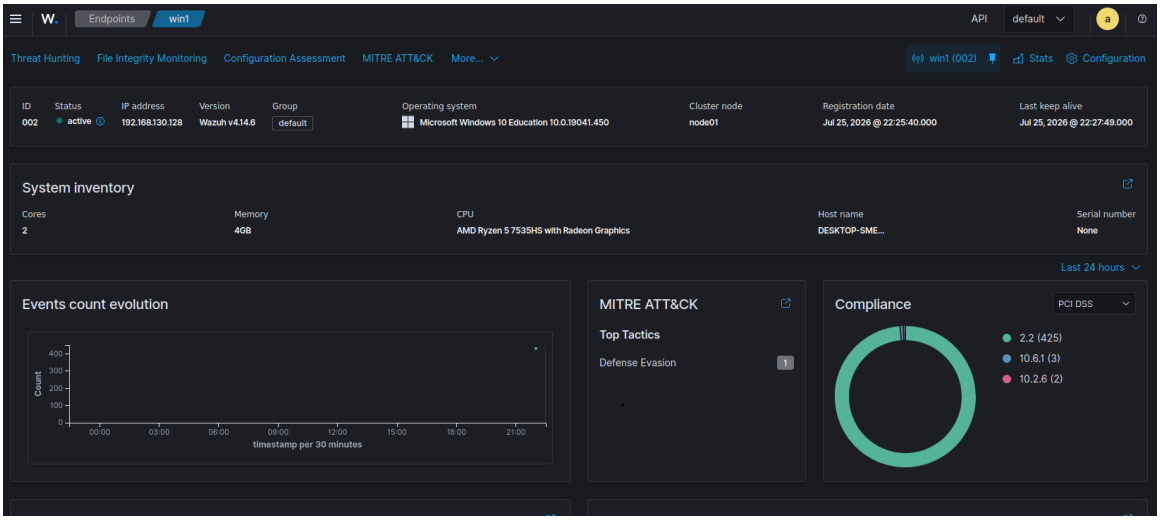


Figure 4 — Endpoint detail view for win1, confirming inventory collection and MITRE ATT&CK / compliance widgets are active.

5. Endpoint Configuration

Basic host-level configuration was also verified on the Ubuntu agent, including DNS-level content restrictions applied via /etc/hosts, confirming the agent host was configured and reachable as expected prior to the attack simulation.

```
ubuntu@wazuh-agent1:~$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       ubuntu2004.linuxvmimages.local  ubuntu2004

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
127.0.0.1 youtube.com # wazuh-block-youtube
127.0.0.1 www.youtube.com # wazuh-block-youtube
127.0.0.1 m.youtube.com # wazuh-block-youtube
ubuntu@wazuh-agent1:~$
```

Figure 5 — /etc/hosts configuration on the Ubuntu Wazuh agent host.

6. Lab User Provisioning

Two lab accounts were provisioned on the Ubuntu agent to support the exercise: ag1 (“Gon”), used as the initial access point, and ag2 (“Kilua”), created afterward as a secondary account. An SSH session was first established as ag1.

```
ag1@wazuh-agent1: ~
ubuntu@wazuh-agent1:~$ su - ag1
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ag1@wazuh-agent1:~$ ssh ag1@192.168.130.183
The authenticity of host '192.168.130.183 (192.168.130.183)' can't be esta
blished.
ECDSA key fingerprint is SHA256:QFt4WnFMqj+7Z78O90K19MjerV+8YdOAaSibiq4MQf
o.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.130.183' (ECDSA) to the list of known
hosts.
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
                        LINUXVMIMAGES.COM
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
                   User Name: ubuntu
                   Password:  ubuntu (sudo su -)
ag1@192.168.130.183's password:
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
```

Figure 6 — Interactive SSH login as ag1 (“Gon”) on the Ubuntu agent host.

The second account, ag2 (“Kilua”), was then created with adduser and its entry confirmed in /etc/passwd.

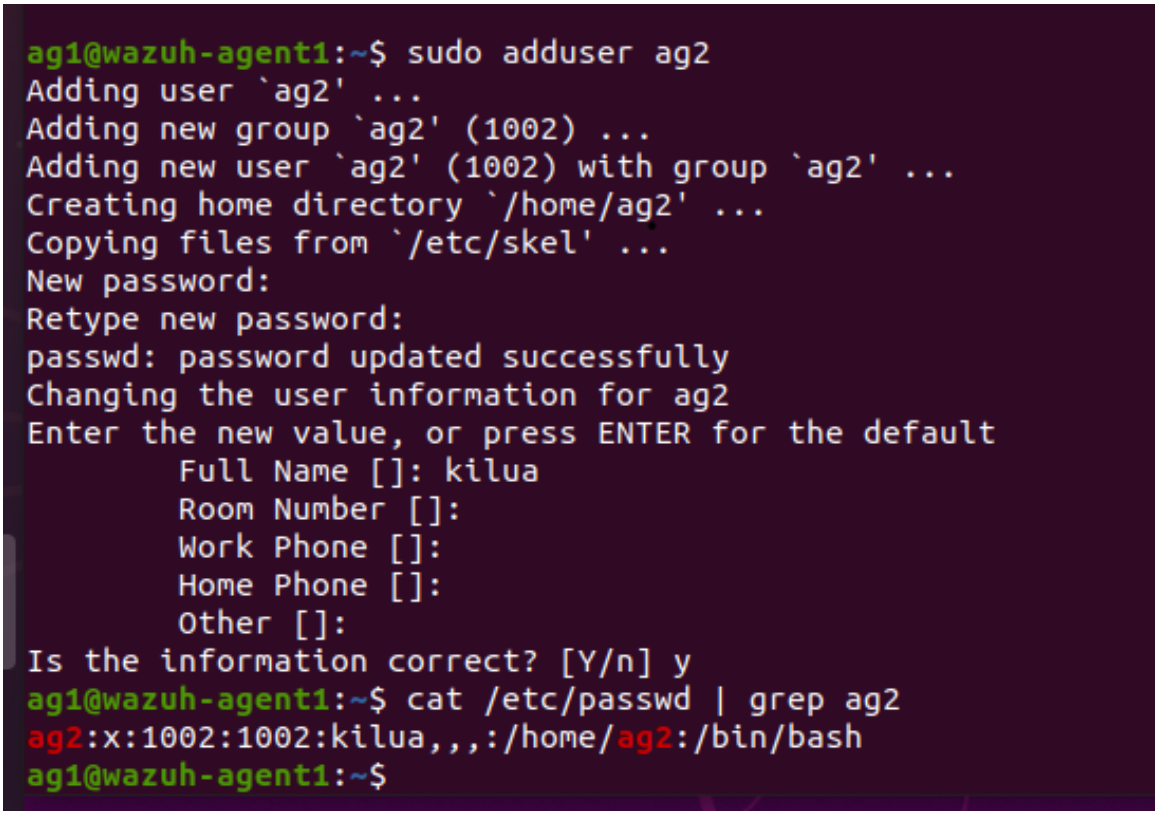


Figure 7 — Creation of the ag2 (“Kilua”) account on the Ubuntu agent.

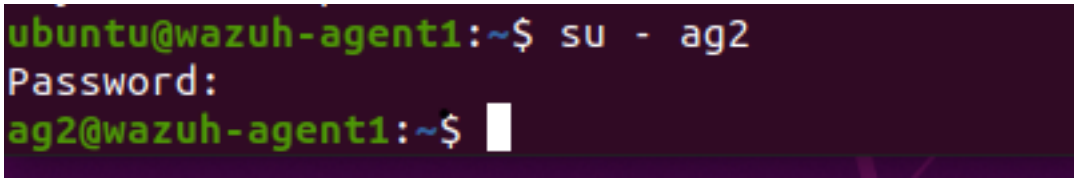


Figure 8 — Switching to the newly created ag2 (“Kilua”) session.

7. Attack Simulation

With the environment enrolled and monitored, a real attack was launched from a Kali Linux machine (192.168.130.129) against the Wazuh manager host (192.168.130.149), to generate authentic detection data rather than synthetic test alerts.

7.1 Reconnaissance

An Nmap service scan was run first to fingerprint open ports and running services on the target, revealing SSH (22), HTTPS (443, the Wazuh dashboard), and the Wazuh API port (55000).

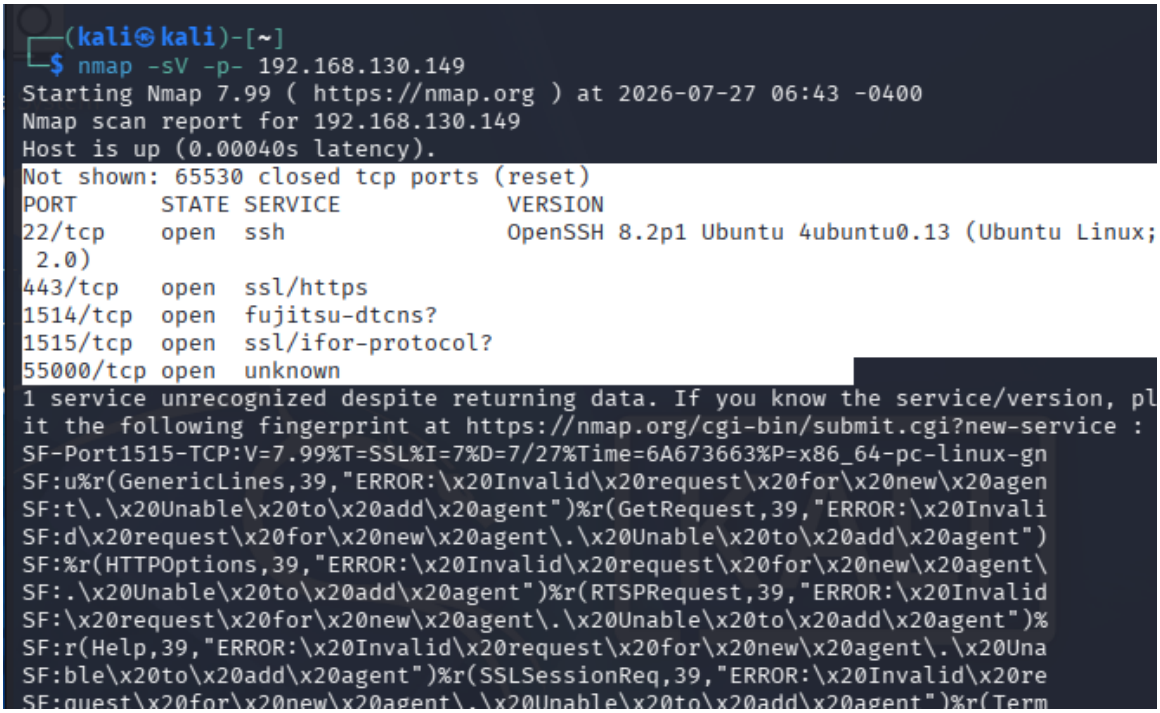


Figure 9 — Nmap service/version scan (nmap -sV -p-) against the target host from the Kali attacker machine.

7.2 SSH Brute-Force

Hydra was used to brute-force SSH credentials for the user1 account using the rockyou wordlist, successfully recovering a valid password after exhausting the candidate list.

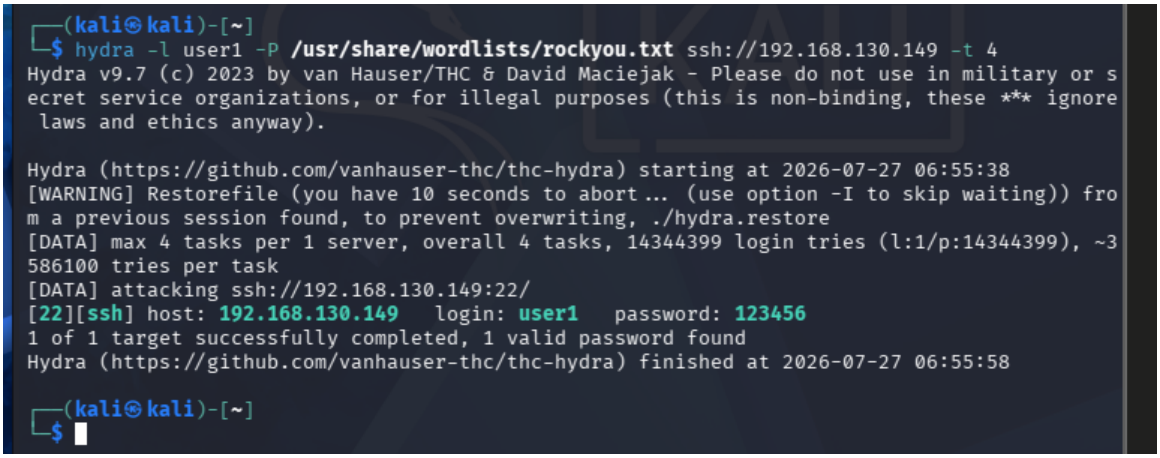


Figure 10 — Hydra SSH brute-force attack against user1, yielding a valid credential.

7.3 Web Dashboard Brute-Force

The attack then pivoted to the Wazuh web dashboard login form (port 443), first with the full rockyou wordlist and then with a smaller targeted common-credentials list, both of which returned multiple valid admin passwords.

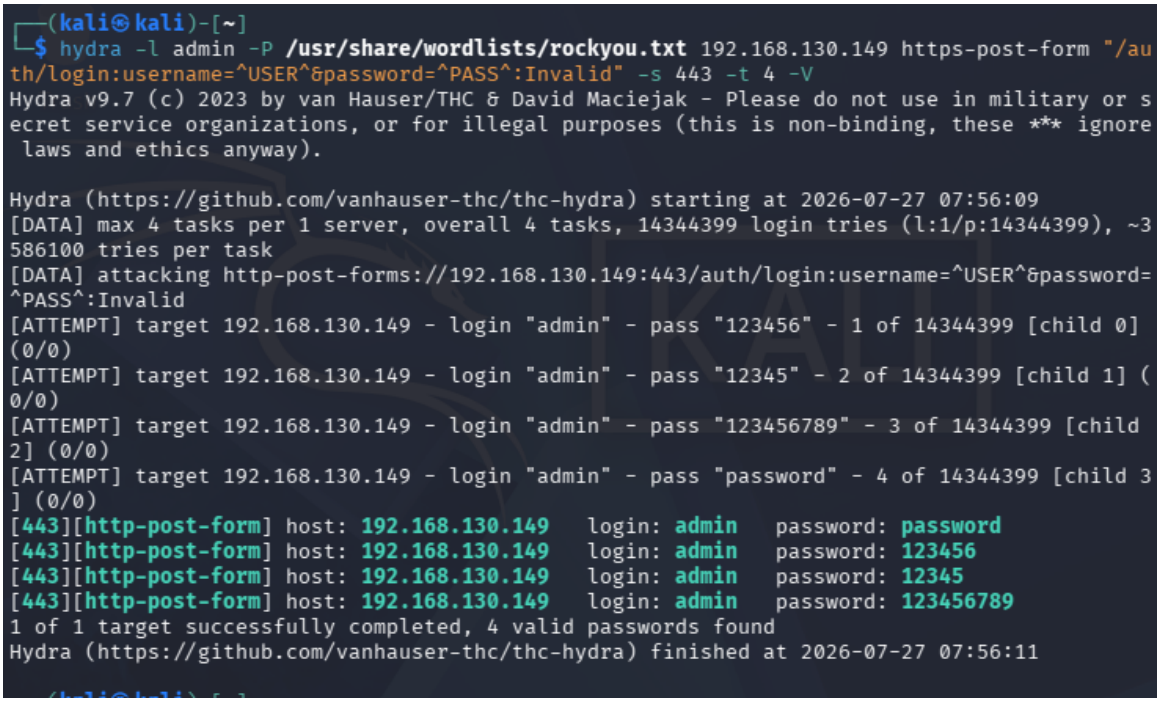


Figure 11 — Hydra HTTP POST-form brute-force against the Wazuh dashboard login, using the rockyou wordlist.

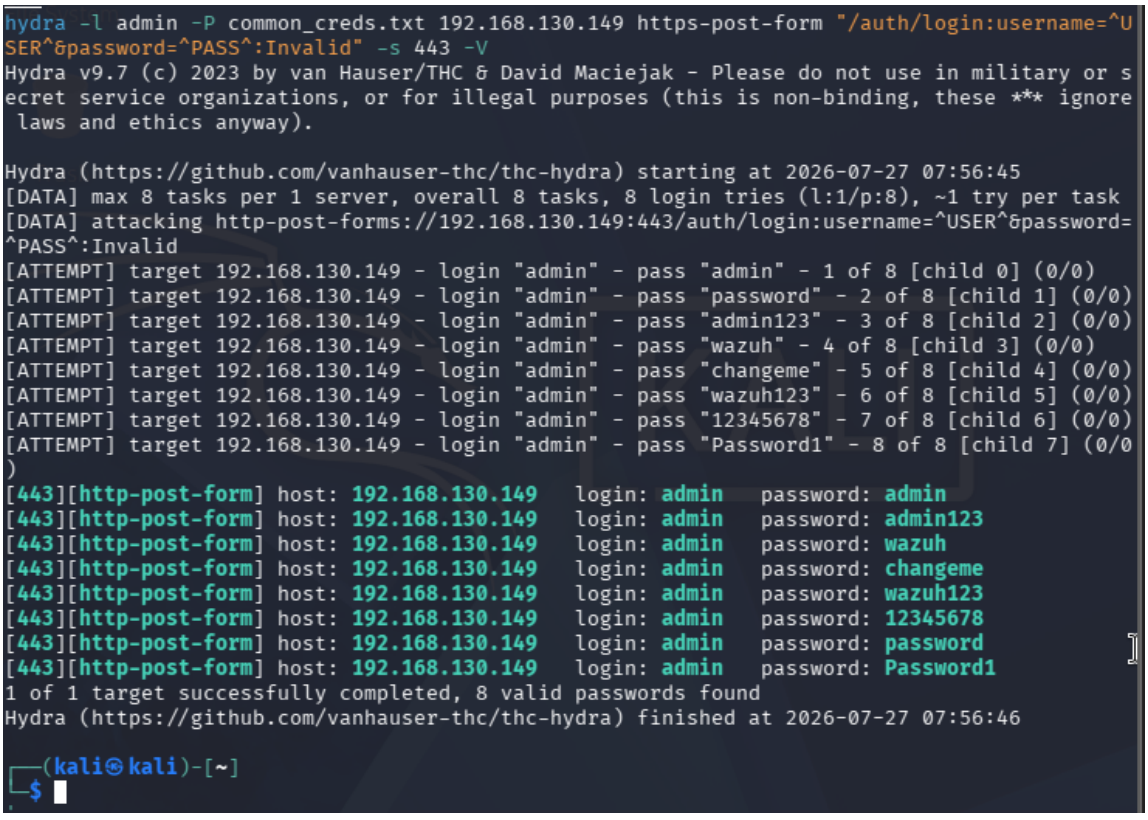


Figure 12 — Follow-up brute-force using a targeted common-credentials list, returning several additional valid admin passwords.

8. Detection and Alerting

Independently of the offensive activity, the Ubuntu host's own alerting surfaced a suspicious privileged command attempt by user1 (an unauthorized sudo attempt), delivered by email — illustrating that

detection was active on the endpoint itself, not only at the SSH/web layer.

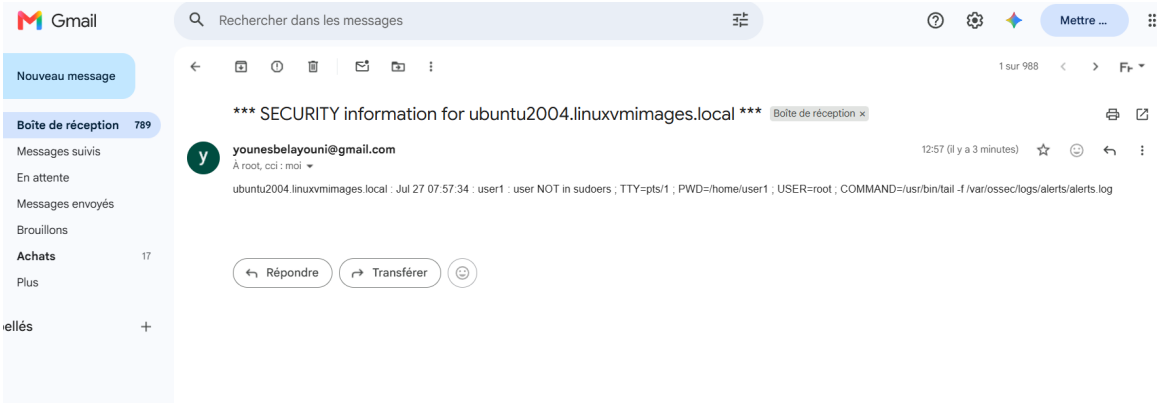


Figure 13 — Email notification for a privileged-command (sudo) attempt by user1 who was not in the sudoers file.

In the Wazuh Threat Hunting dashboard, filtering on the authentication_failed rule group over the attack window showed 512 total matching events, confirming the brute-force traffic was being logged and normalized correctly.

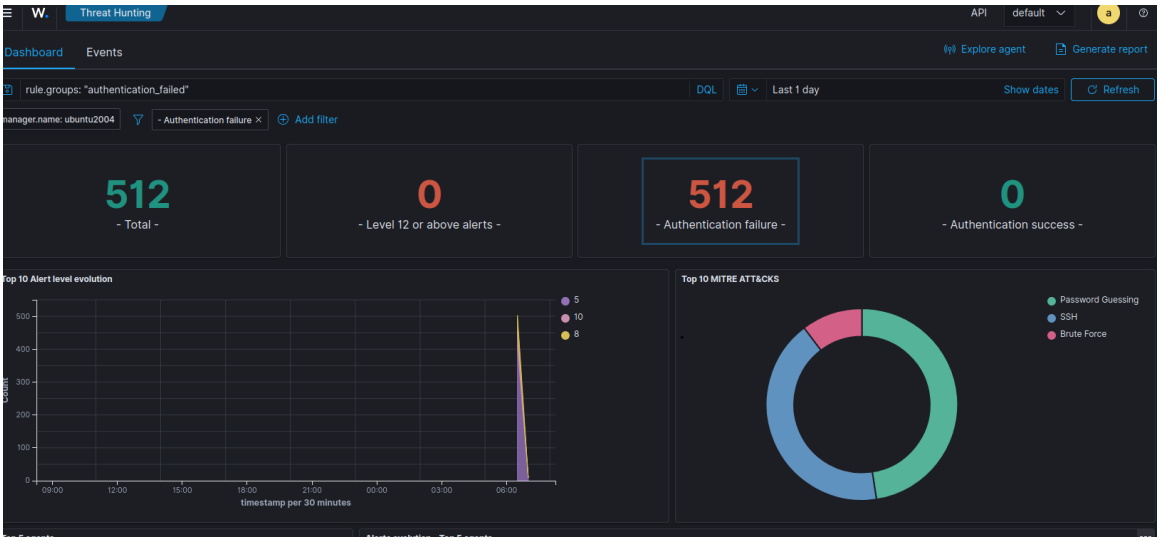


Figure 14 — Threat Hunting dashboard filtered on rule.groups: "authentication_failed", showing 512 matching events.

Narrowing further to high-severity correlation alerts (rule.level ≥ 10) reduced the set to 52 alerts, all classified under the Brute Force MITRE ATT&CK technique, clustered tightly within a single time window — the signature of an automated brute-force tool rather than manual login attempts.

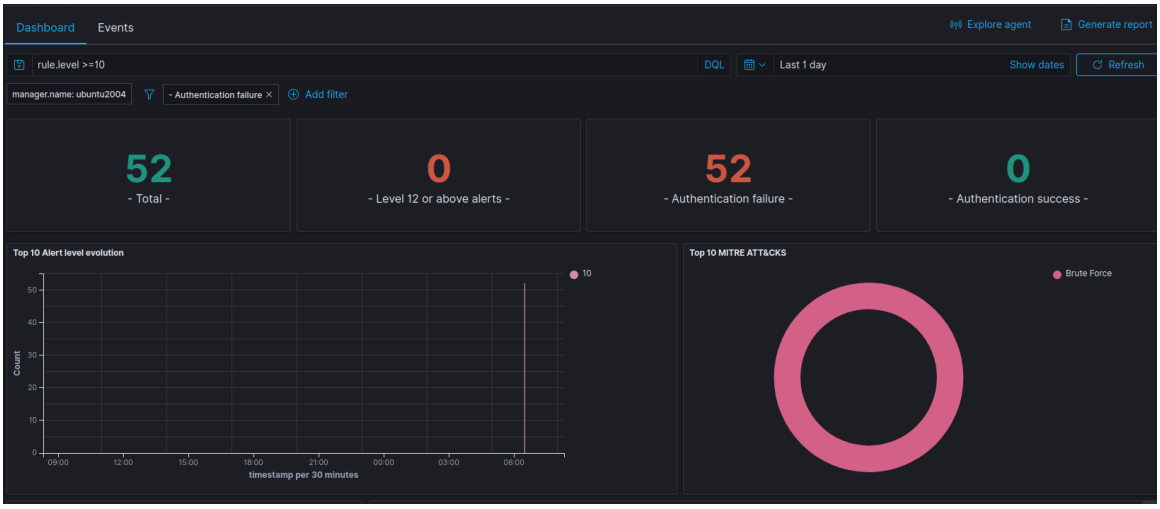


Figure 15 — High-severity alerts (rule.level ≥ 10) mapped to the MITRE ATT&CK Brute Force technique, 52 events total.

9. Investigation — Threat Hunting

Following standard SOC triage practice, the investigation started from the anomaly rather than from prior knowledge of the attacker. The Events view listed each individual alert underlying the 52-alert cluster, all decoded as repeated failed password attempts (rule ID 2502, level 10).

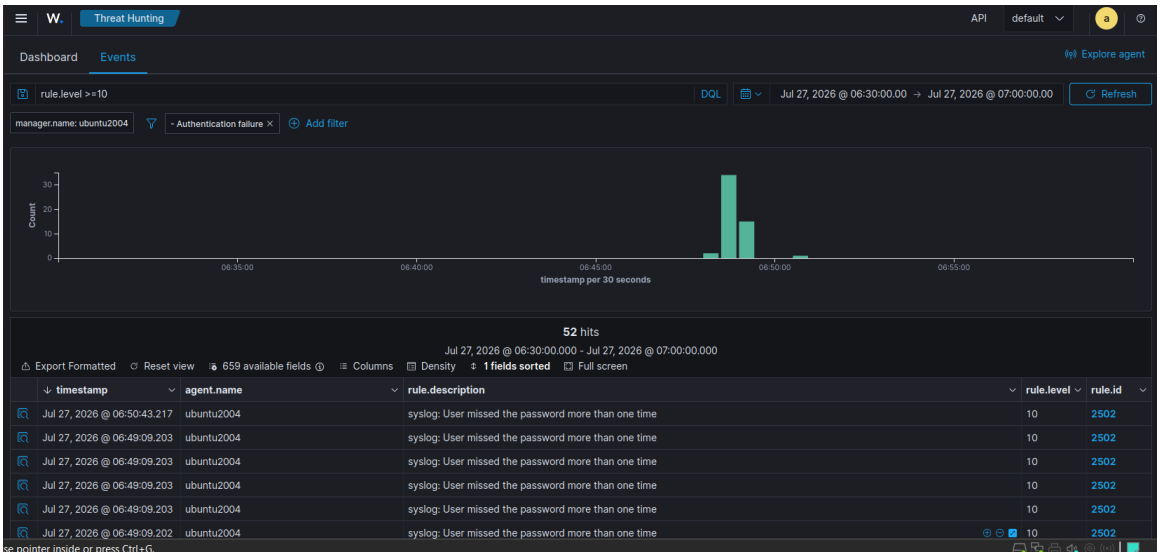


Figure 16 — Events table listing the individual high-severity authentication-failure alerts within the attack window.

Expanding a single event exposed the full field set for that alert. This is the point in the investigation where the attacker was actually identified: the data.srcip field showed 192.168.130.129, with data.srcuser = ag1 and the raw log confirming an invalid-user SSH attempt from that address.

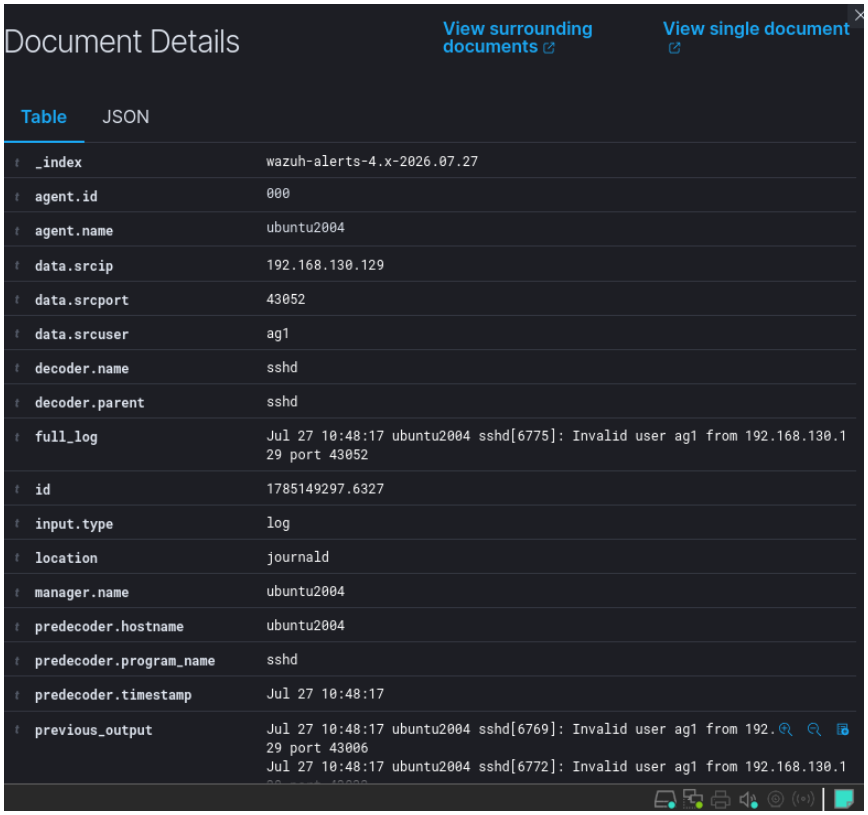


Figure 17 — Expanded event detail identifying the attacker source IP (data.srcip = 192.168.130.129) and targeted account (ag1).

Investigation outcome

Attacker source IP: 192.168.130.129 | Technique: T1110 Brute Force | Targeted account: ag1 | Peak severity: level 10

10. Incident Response — Containment

With the attacker IP confirmed, a UFW firewall rule was applied on the Ubuntu host to block all further inbound traffic from that address.

```
sudo ufw deny from 192.168.130.129
sudo ufw reload
```

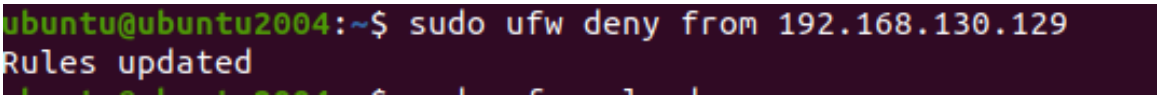


Figure 18 — UFW deny rule applied against the identified attacker IP.

The rule set was then verified, confirming an active DENY IN entry for 192.168.130.129 ahead of any broader allow rules, while the SSH allow rule for legitimate access remained intact.

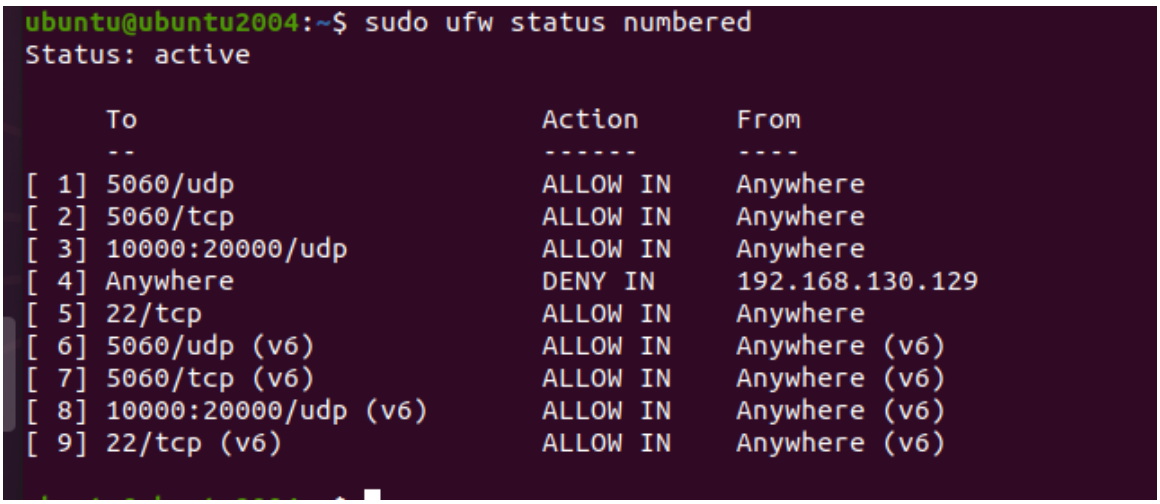


Figure 19 — Verified UFW rule set showing the attacker IP explicitly denied, with legitimate SSH access preserved.

11. Results and Evaluation

The lab successfully validated a complete, evidenced attack-to-response chain:

- Reconnaissance and brute-force traffic from a real attacker machine was generated and captured (Nmap + Hydra, SSH and web vectors).
- Wazuh correctly normalized and correlated the resulting failed-login events, escalating 52 of 512 raw events to high severity.
- Alerts were automatically mapped to the MITRE ATT&CK Brute Force technique without manual classification.
- The attacker's source IP (192.168.130.129) was identified through standard alert-pivoting, following anomaly-first triage rather than prior knowledge.
- The threat was contained with a verified firewall rule, completing the detection-to-response loop.

This confirms the platform performs its intended function end to end: ingest, normalize, detect, classify, investigate, and contain — even though the enrichment (VirusTotal/AbuseIPDB) and SOAR automation layers from the original design remain a planned extension rather than an implemented one at this stage.

12. Conclusion

This project demonstrates the practical deployment of a functional mini-SOC using Wazuh, validated against a real, self-generated attack rather than synthetic test data. Beyond basic log collection, it demonstrates endpoint enrollment across operating systems, detection engineering, MITRE ATT&CK alignment, hands-on SOC triage methodology, and manual incident containment — core competencies expected of SOC analysts and security engineers. Extending the platform with automated threat-intelligence enrichment and SOAR-driven response, as outlined in the original architecture, is the natural next step for this lab.

References

- Wazuh Documentation — https://documentation.wazuh.com
- MITRE ATT&CK Framework — https://attack.mitre.org
- Nmap Reference Guide — https://nmap.org

- THC-Hydra — https://github.com/vanhauser-thc/thc-hydra